

Python for Beginners

A Complete Guide from Zero to Your First Real Projects

Covers installation, syntax, data types, control flow, functions, data structures, object-oriented programming, file handling, error handling, modules, and hands-on projects.

Table of Contents

Chapter 1	Getting Started — Installing Python and Writing Your First Program
Chapter 2	Variables, Data Types, and Basic Operators
Chapter 3	Strings in Depth
Chapter 4	Control Flow — Conditionals and Loops
Chapter 5	Functions
Chapter 6	Data Structures — Lists, Tuples, Dictionaries, Sets
Chapter 7	Working with Files
Chapter 8	Error Handling and Exceptions
Chapter 9	Object-Oriented Programming
Chapter 10	Modules, Packages, and the Standard Library
Chapter 11	Iterators, Generators, and Functional Tools
Chapter 12	Debugging, Testing, and Writing Clean Code
Chapter 13	Mini Projects — Putting It All Together
Chapter 14	Where to Go Next

This guide is meant to be read in order — each chapter builds on ideas from the last. Type out every code example yourself rather than just reading it; typing builds muscle memory that reading alone never will.

Getting Started

What is Python, and why learn it?

Python is a general-purpose programming language known for being readable, beginner-friendly, and extremely versatile. It is used to build websites, automate boring tasks, analyze data, train machine learning models, control hardware, and much more. Compared to many other languages, Python reads almost like plain English, which makes it an excellent first language.

- **Readable syntax** — Python code is designed to be easy to read and write.
- **Huge ecosystem** — libraries exist for nearly every task imaginable.
- **Interpreted** — you run code directly, without a separate compile step.
- **Widely used** — in web development, data science, automation, and education.

Installing Python

Most modern computers do not come with a recent version of Python pre-installed (or the version installed is too old). Here is how to get it:

Windows

- 1 Go to python.org/downloads and download the latest Python 3 installer.
- 2 Run the installer. Check the box that says "Add Python to PATH" — this step trips up most beginners if skipped.
- 3 Click Install Now and wait for it to finish.

macOS

- 1 Go to python.org/downloads and download the macOS installer, or
- 2 If you use Homebrew, open Terminal and run: `brew install python`

Linux

Most Linux distributions ship with Python 3 already installed. Open a terminal and check with the command below. If it's missing, install it with your package manager (e.g. `sudo apt install python3` on Ubuntu/Debian).

Verifying your installation

Open a terminal (Command Prompt, Terminal, or PowerShell) and type:

```
python --version
```

```
# or on some systems:
python3 --version
```

You should see something like Python 3.12.1. If you see an error, revisit the installation step above.

NOTE: Throughout this guide, when a command doesn't work with `python`, try `python3` instead. Some systems keep an old Python 2 installation around under the plain `python` name.

Choosing where to write code

You need a text editor or IDE (Integrated Development Environment) to write Python code. Popular beginner-friendly choices:

Tool	Best for
VS Code	Free, lightweight, huge extension ecosystem — most popular overall choice
PyCharm Community	Full-featured IDE built specifically for Python
Thonny	Extremely simple, built for absolute beginners
Jupyter Notebook	Great for data analysis and experimenting cell-by-cell

TIP: If you're not sure where to start, install VS Code and add the official "Python" extension from Microsoft. It's free and used by professionals and beginners alike.

Your first program

By tradition, the first program in any language prints "Hello, World!" to the screen. Create a new file named `hello.py` and type:

```
print("Hello, World!")
```

Save the file, open a terminal in the same folder, and run:

```
python hello.py
```

You should see:

```
Hello, World!
```

Congratulations — you just wrote and ran your first Python program. Let's break down what happened:

- `print()` is a built-in function that displays text on the screen.
- The text "Hello, World!" is called a string — text data, always wrapped in quotes.
- Python read your file top to bottom and executed each line.

The interactive shell (REPL)

Python also comes with an interactive mode where you can type code and see results immediately. This is useful for quick experiments. Open a terminal and type `python` with no filename:

```
>>> print("Hello!")
Hello!
>>> 2 + 2
4
>>> exit()
```

Type `exit()` or press `Ctrl+D` (`Ctrl+Z` on Windows) to leave.

Comments

Comments are notes in your code that Python ignores when running. Use them to explain your reasoning to your future self and to others reading your code.

```
# This is a single-line comment
print("This line runs") # comments can also follow code

"""
This is a multi-line comment,
technically a string that isn't assigned to anything.
"""
```

Chapter Summary

- Python is installed and verified with `python --version`.
- Code is written in `.py` files and run from the terminal.
- `print()` displays output.
- Comments start with `#`.

Try It Yourself — Chapter 1

- 1 Write a program that prints your name, age, and favorite hobby on three separate lines.
- 2 Open the interactive shell and use it as a calculator to compute the area of a rectangle with width 12 and height 5.
- 3 Write a program with a comment above each line explaining, in your own words, what that line does.

Variables, Data Types, and Basic Operators

What is a variable?

A variable is a name that refers to a value stored in memory. Think of it as a labeled box you can put data into, look at, and change later.

```
age = 25
name = "Maria"
is_student = True

print(age)
print(name)
print(is_student)
```

Python figures out the type of a variable automatically based on the value you assign — this is called dynamic typing. You don't have to declare types up front like in some other languages.

Naming rules

- Names can contain letters, numbers, and underscores, but can't start with a number.
- Names are case-sensitive: `age` and `Age` are different variables.
- By convention, Python variables use `snake_case` (e.g. `first_name`), not `camelCase`.
- Avoid Python's reserved keywords (like `class`, `for`, `if`) as variable names.

Core data types

Type	Example	Description
<code>int</code>	<code>42</code> , <code>-7</code> , <code>1000</code>	Whole numbers
<code>float</code>	<code>3.14</code> , <code>-0.5</code>	Decimal numbers
<code>str</code>	<code>"hello"</code>	Text
<code>bool</code>	<code>True</code> , <code>False</code>	Boolean logic values
<code>NoneType</code>	<code>None</code>	Represents "no value"

Use the built-in `type()` function to check a variable's type:

```
x = 5
print(type(x))    # <class 'int'>

y = 5.0
print(type(y))    # <class 'float'>
```

```
z = "5"
print(type(z))    # <class 'str'>
```

NOTE: 5 and "5" are not the same. The first is a number; the second is text that happens to look like a number. This trips up almost every beginner at some point.

Arithmetic operators

Operator	Meaning	Example	Result
+	Addition	5 + 3	8
-	Subtraction	5 - 3	2
*	Multiplication	5 * 3	15
/	Division (always returns float)	5 / 2	2.5
//	Floor division (drops remainder)	5 // 2	2
%	Modulus (remainder)	5 % 2	1
**	Exponentiation	5 ** 2	25

Type conversion

You can convert between types using `int()`, `float()`, and `str()`:

```
age_text = "25"
age_number = int(age_text)
print(age_number + 5)    # 30

price = 19.99
price_text = str(price)
print("Price: " + price_text)
```

Getting input from the user

The `input()` function pauses your program and waits for the user to type something. It always returns a string, so convert it if you need a number.

```
name = input("What is your name? ")
print("Hello, " + name + "!")

age = int(input("How old are you? "))
print("Next year you will be", age + 1)
```

f-strings — the modern way to format text

f-strings let you embed variables directly inside a string by prefixing it with `f` and wrapping variables in curly braces. This is the most common and readable way to build strings in modern Python.

```
name = "Alex"
age = 30
print(f"{name} is {age} years old.")
print(f"Next year, {name} will be {age + 1}.")
```

TIP: Prefer f-strings over string concatenation with + — they're easier to read and less error-prone, especially with numbers.

Comparison and logical operators

Operator	Meaning
==	Equal to
!=	Not equal to
> / <	Greater than / less than
>= / <=	Greater than or equal / less than or equal
and	True if both sides are True
or	True if at least one side is True
not	Flips True/False

```
age = 20
print(age >= 18 and age < 65)  # True
print(age == 21)                # False
```

Chapter Summary

- Variables store data and are created with =.
- Core types: int, float, str, bool, None.
- f-strings (f"(...)") are the cleanest way to format text with variables.
- input() always returns a string.

Try It Yourself — Chapter 2

- 1 Write a program that asks for two numbers and prints their sum, difference, product, and quotient.
- 2 Write a program that converts a temperature from Fahrenheit to Celsius, entered by the user.
- 3 Ask the user for their birth year and print how old they will turn this year, using an f-string.
- 4 Predict the output of `7 // 2`, `7 % 2`, and `7 ** 2` before running them to check yourself.

Strings in Depth

Creating strings

```
single = 'Hello'
double = "Hello"
multi = """This string
spans multiple
lines."""
```

Single and double quotes work the same way — pick one style and stay consistent. Triple quotes let a string span multiple lines, and are also used for docstrings (documentation comments).

Indexing and slicing

Strings are sequences of characters, and each character has a numeric position called an index, starting at 0.

```
word = "Python"
print(word[0])      # P
print(word[5])      # n
print(word[-1])     # n (last character)
print(word[0:3])    # Pyt (characters 0,1,2 – end index excluded)
print(word[:2])     # Py
print(word[2:])     # thon
```

Common string methods

Method	What it does	Example
<code>.upper()</code>	Converts to uppercase	<code>"hi".upper()</code> → <code>"HI"</code>
<code>.lower()</code>	Converts to lowercase	<code>"HI".lower()</code> → <code>"hi"</code>
<code>.strip()</code>	Removes leading/trailing whitespace	<code>" hi ".strip()</code> → <code>"hi"</code>
<code>.replace(a, b)</code>	Replaces all occurrences	<code>"cat".replace("c", "b")</code> → <code>"bat"</code>
<code>.split(sep)</code>	Splits into a list	<code>"a,b,c".split(",")</code> → <code>['a', 'b', 'c']</code>
<code>.join(list)</code>	Joins a list into a string	<code>"-".join(['a', 'b'])</code> → <code>"a-b"</code>
<code>.find(sub)</code>	Returns index of first match, or -1	<code>"hello".find("l")</code> → <code>2</code>
<code>.startswith(x)</code>	True/False check	<code>"hello".startswith("he")</code> → <code>True</code>
<code>.format()</code> / f-string	Inserts values into text	see Chapter 2

Useful patterns

```
sentence = "    The Quick Brown Fox    "  
clean = sentence.strip().lower()  
print(clean)           # "the quick brown fox"  
  
words = clean.split()  
print(words)           # ['the', 'quick', 'brown', 'fox']  
print(len(words))      # 4
```

String concatenation and repetition

```
greeting = "Hi" + " " + "there"  
print(greeting)       # Hi there  
  
line = "-" * 20  
print(line)           # -----
```

Checking what's inside a string

```
text = "hello world"  
print("world" in text) # True  
print("bye" in text)   # False
```

NOTE: Strings are immutable — you cannot change a character in place (`word[0] = "J"` raises an error). String methods always return a new string rather than modifying the original.

Chapter Summary

- Strings are indexed from 0, and support slicing with `[start:end]`.
- Common methods: `.upper()` `.lower()` `.strip()` `.split()` `.join()` `.replace()`.
- Strings are immutable — methods return new strings.

Try It Yourself — Chapter 3

- 1 Write a program that takes a full name and prints just the first name and just the last name.
- 2 Write a program that checks whether a word entered by the user is a palindrome (reads the same forwards and backwards). Hint: compare the string to its reverse, `word[::-1]`.
- 3 Given the sentence "the quick brown fox", use `.split()` and `" ".join()` to reverse the order of the words.
- 4 Write a function that counts how many vowels are in a given string.

Control Flow — Conditionals and Loops

if, elif, else

Conditional statements let your program make decisions and run different code depending on circumstances. Python uses indentation (spaces) — not curly braces — to define code blocks.

```
age = 20
if age < 13:
    print("Child")
elif age < 20:
    print("Teenager")
else:
    print("Adult")
```

NOTE: Indentation is not optional in Python — it's how the language knows which lines belong to which block. The standard is 4 spaces per indent level. Mixing tabs and spaces will cause errors.

Truthy and falsy values

Every value in Python is considered either "truthy" or "falsy" when used in a boolean context. Falsy values include: 0, 0.0, "" (empty string), [] (empty list), None, and False itself. Everything else is truthy.

```
name = ""
if name:
    print("Hello,", name)
else:
    print("No name given")
```

while loops

A while loop repeats a block of code as long as a condition remains true.

```
count = 0
while count < 5:
    print(count)
    count += 1
print("Done!")
```

break and continue

```
n = 0
while True:
    n += 1
    if n == 3:
        continue # skip the rest of this iteration
    if n > 5:
        break # exit the loop entirely
```

```
print(n)
```

for loops

A for loop iterates over a sequence — such as a string, list, or range of numbers.

```
for letter in "Python":
    print(letter)

for number in range(5):
    print(number)           # 0 1 2 3 4

for number in range(2, 10, 2):
    print(number)          # 2 4 6 8
```

range() call	Produces
range(5)	0, 1, 2, 3, 4
range(2, 6)	2, 3, 4, 5
range(0, 10, 2)	0, 2, 4, 6, 8
range(10, 0, -1)	10, 9, 8, ... 1

Looping over lists with index

```
fruits = ['apple', 'banana', 'cherry']
for fruit in fruits:
    print(fruit)

# with index, using enumerate
for index, fruit in enumerate(fruits):
    print(index, fruit)
```

Nested loops and conditionals

```
for row in range(3):
    for col in range(3):
        if row == col:
            print("*", end=" ")
        else:
            print(".", end=" ")
    print()
```

TIP: If a loop or if-statement is getting hard to read because of deep nesting, that's often a sign you should break the logic into a separate function — covered next in Chapter 5.

Chapter Summary

- if / elif / else makes decisions; indentation defines blocks.
- while loops repeat based on a condition; for loops iterate over sequences.
- break exits a loop early; continue skips to the next iteration.
- range(start, stop, step) generates number sequences.

Try It Yourself — Chapter 4

- 1 Write a program that prints all numbers from 1 to 50 that are divisible by 3 or 5.
- 2 Write a program that asks the user to guess a number between 1 and 10, giving them 3 tries before revealing the answer.
- 3 Print a multiplication table for a number the user provides, from 1x to 10x, using a for loop.
- 4 Use nested loops to print a triangle of asterisks, 5 rows tall, growing by one star per row.

Functions

Why functions?

Functions let you package a block of code under a name so you can reuse it without retyping it. They make programs shorter, easier to test, and easier to reason about.

Defining and calling a function

```
def greet(name):  
    print(f"Hello, {name}!")  
  
greet("Sam")  
greet("Priya")
```

Return values

Functions can send a value back to the caller using `return`. Without an explicit return, a function returns `None`.

```
def add(a, b):  
    return a + b  
  
result = add(3, 4)  
print(result)    # 7
```

Default arguments

```
def greet(name, greeting="Hello"):  
    print(f"{greeting}, {name}!")  
  
greet("Sam")                # Hello, Sam!  
greet("Sam", "Hi")          # Hi, Sam!  
greet("Sam", greeting="Hey") # Hey, Sam!
```

Keyword arguments and `*args` / `**kwargs`

You can pass arguments by name for clarity, and write functions that accept a variable number of arguments.

```
def describe_pet(name, animal="dog"):  
    print(f"{name} is a {animal}")  
  
describe_pet(animal="cat", name="Whiskers")  
  
def total(*numbers):  
    return sum(numbers)
```

```
print(total(1, 2, 3, 4))    # 10

def show_info(**details):
    for key, value in details.items():
        print(f"{key}: {value}")

show_info(name="Amy", age=28)
```

Variable scope

Variables created inside a function only exist inside that function — this is called local scope. Variables created outside all functions are in the global scope.

```
def my_function():
    x = 10    # local variable
    print(x)

my_function()
# print(x)    # this would raise a NameError – x doesn't exist out here
```

NOTE: Avoid relying on global variables inside functions where possible — passing values in as arguments and returning results makes functions easier to understand and test in isolation.

Docstrings

A docstring is a short description placed as the first line inside a function, explaining what it does. Good practice, especially as programs grow.

```
def area_of_circle(radius):
    """Return the area of a circle given its radius."""
    return 3.14159 * radius ** 2
```

Lambda functions (short anonymous functions)

A lambda is a small, unnamed function defined in a single line. Useful for short throwaway operations, often passed into other functions.

```
square = lambda x: x ** 2
print(square(5))    # 25

numbers = [5, 2, 8, 1]
numbers.sort(key=lambda n: -n)
print(numbers)      # [8, 5, 2, 1]
```

Chapter Summary

- Functions are defined with `def name(parameters):` and reusable.
- `return` sends a value back; without it, a function returns `None`.
- Default and keyword arguments make functions flexible and calls more readable.
- Variables inside a function are local unless declared otherwise.

Try It Yourself — Chapter 5

- 1 Write a function `is_even(n)` that returns `True` or `False`.
- 2 Write a function `celsius_to_fahrenheit(c)` that returns the converted temperature.
- 3 Write a function that takes a list of numbers and returns the largest one, without using the built-in `max()`.
- 4 Write a function `fizzbuzz(n)` that prints numbers 1 to `n`, but prints "Fizz" for multiples of 3, "Buzz" for multiples of 5, and "FizzBuzz" for both.

Data Structures — Lists, Tuples, Dictionaries, Sets

Lists

A list is an ordered, changeable (mutable) collection of items.

```
fruits = ["apple", "banana", "cherry"]
print(fruits[0])          # apple

fruits.append("date")     # add to end
fruits[1] = "blueberry"   # replace an item
print(fruits)
print(len(fruits))        # 4
```

Common list methods

Method	What it does
<code>.append(x)</code>	Adds x to the end
<code>.insert(i, x)</code>	Inserts x at position i
<code>.remove(x)</code>	Removes the first matching value x
<code>.pop(i)</code>	Removes and returns item at index i (last item if omitted)
<code>.sort()</code>	Sorts the list in place
<code>.reverse()</code>	Reverses the list in place
<code>.index(x)</code>	Returns the index of the first match of x
<code>in</code>	Checks membership: "a" in mylist

List slicing works like string slicing

```
numbers = [10, 20, 30, 40, 50]
print(numbers[1:3])    # [20, 30]
print(numbers[:2])     # [10, 20]
print(numbers[-2:])    # [40, 50]
```

List comprehensions

A concise way to build a new list from an existing sequence. This is one of Python's most distinctive and useful features.

```
squares = [n ** 2 for n in range(6)]
print(squares)    # [0, 1, 4, 9, 16, 25]
```

```
evens = [n for n in range(20) if n % 2 == 0]
print(evens)      # [0, 2, 4, ..., 18]
```

Tuples

A tuple is like a list, but immutable — once created, it can't be changed. Use tuples for fixed collections of values, like coordinates.

```
point = (3, 4)
print(point[0])    # 3

x, y = point        # unpacking
print(x, y)         # 3 4
```

Dictionaries

A dictionary stores data as key-value pairs. Keys must be unique and are typically strings or numbers.

```
person = {"name": "Alex", "age": 30, "city": "Boston"}
print(person["name"])    # Alex

person["age"] = 31        # update a value
person["email"] = "a@x.com" # add a new key
print(person)
```

Common dictionary operations

```
for key in person:
    print(key, person[key])

for key, value in person.items():
    print(f"{key}: {value}")

print(person.get("phone", "Not found")) # safe lookup with a default
print("name" in person)                 # True
```

Sets

A set is an unordered collection of unique values. Great for removing duplicates and membership testing.

```
numbers = [1, 2, 2, 3, 3, 3, 4]
unique_numbers = set(numbers)
print(unique_numbers)    # {1, 2, 3, 4}

a = {1, 2, 3}
b = {2, 3, 4}
print(a | b)    # union: {1, 2, 3, 4}
print(a & b)    # intersection: {2, 3}
print(a - b)    # difference: {1}
```

Nesting data structures

Real-world data is often nested — lists of dictionaries, dictionaries containing lists, and so on. This is extremely common when working with JSON data from web APIs.

```
students = [
    {"name": "Alex", "grades": [85, 92, 78]},
    {"name": "Priya", "grades": [90, 95, 88]},
]

for student in students:
    average = sum(student["grades"]) / len(student["grades"])
    print(f"{student['name']}: {average:.1f}")
```

Here, `students` is a list, each item is a dictionary, and each dictionary contains a list of numbers under the `"grades"` key. You access nested values by chaining `[]` lookups: `students[0]["grades"][1]` gives Alex's second grade, 92.

TIP: When a nested structure gets hard to read on one line, format it across multiple lines like the example above — Python doesn't care about the extra whitespace inside brackets, and it makes the structure much easier for humans to follow.

Choosing the right structure

Structure	Ordered?	Mutable?	Duplicates?	Use when...
list	Yes	Yes	Yes	You need an ordered, changeable sequence
tuple	Yes	No	Yes	Data shouldn't change (fixed records)
dict	Yes*	Yes	Keys: No	You need labeled/key-based lookup
set	No	Yes	No	You need uniqueness or fast membership checks

*Dictionaries preserve insertion order since Python 3.7.

Chapter Summary

- Lists: ordered, mutable, use `[]`.
- Tuples: ordered, immutable, use `()`.
- Dictionaries: key-value pairs, use `{}` with `key: value`.
- Sets: unordered, unique values, use `{}` or `set()`.
- List comprehensions offer a concise way to transform sequences.

Try It Yourself — Chapter 6

- 1 Write a program that removes duplicate values from a list while preserving order.
- 2 Build a dictionary that maps 5 country names to their capital cities, then write a loop that prints each pair as "The capital of X is Y".
- 3 Given a list of numbers, use a list comprehension to build a new list containing only the numbers greater than 10.
- 4 Write a function that takes two lists and returns a dictionary pairing items from the first list (as keys) with items from the second (as values), using `zip()`.

Working with Files

Opening and reading files

The safest way to work with files in Python is using a `with` block, which automatically closes the file even if an error occurs.

```
with open("notes.txt", "r") as file:
    contents = file.read()
    print(contents)
```

Mode	Meaning
"r"	Read (default) — file must exist
"w"	Write — creates file if missing, overwrites if it exists
"a"	Append — adds to the end of the file
"r+"	Read and write

Reading line by line

```
with open("notes.txt", "r") as file:
    for line in file:
        print(line.strip()) # strip() removes the trailing newline

# or read all lines into a list
with open("notes.txt", "r") as file:
    lines = file.readlines()
```

Writing to files

```
with open("output.txt", "w") as file:
    file.write("First line\n")
    file.write("Second line\n")

with open("output.txt", "a") as file:
    file.write("This gets appended\n")
```

Working with CSV files

CSV (comma-separated values) is a common format for tabular data. Python's built-in `csv` module handles it well.

```
import csv
```

```

with open("people.csv", "r") as file:
    reader = csv.reader(file)
    for row in reader:
        print(row)

with open("output.csv", "w", newline="") as file:
    writer = csv.writer(file)
    writer.writerow(["name", "age"])
    writer.writerow(["Alex", 30])

```

Working with JSON

JSON is a common format for structured data, widely used by web APIs. Python's built-in json module converts between JSON text and Python dictionaries/lists.

```

import json

data = {"name": "Alex", "age": 30, "hobbies": ["chess", "hiking"]}

# write JSON to a file
with open("data.json", "w") as file:
    json.dump(data, file)

# read JSON from a file
with open("data.json", "r") as file:
    loaded = json.load(file)

print(loaded["name"])    # Alex

```

NOTE: File paths can be relative (based on where you run your script) or absolute (the full path starting from the drive/root). If a file "cannot be found", the most common cause is running the script from a different folder than expected.

Working with paths using pathlib

The pathlib module offers a modern, more readable way to work with file paths than manually joining strings, and works consistently across Windows, macOS, and Linux.

```

from pathlib import Path

folder = Path("data")
file_path = folder / "notes.txt"    # builds a path safely, any OS

print(file_path.exists())    # True/False
print(file_path.name)        # notes.txt
print(file_path.suffix)      # .txt
print(file_path.parent)      # data

for txt_file in folder.glob("*.txt"):
    print(txt_file)    # lists every .txt file in the folder

```

Chapter Summary

- Always use `with open(...)` as file: to handle files safely.
- Modes: "r" read, "w" write (overwrite), "a" append.
- Use the `csv` and `json` modules for structured data formats.

Try It Yourself — Chapter 7

- 1 Write a program that creates a text file, writes 5 lines to it, then reads and prints those lines back.
- 2 Write a program that counts the number of lines and words in a text file.
- 3 Create a small CSV file of 3 people with name and age columns, then write a program that reads it and prints the average age.
- 4 Store a Python dictionary of settings (e.g. volume, brightness, username) as JSON, then write a program that loads it back and prints each setting.

Error Handling and Exceptions

What is an exception?

When something goes wrong while your program runs — dividing by zero, opening a missing file, converting invalid text to a number — Python raises an exception. If unhandled, the program crashes and prints a traceback.

```
>>> 10 / 0
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ZeroDivisionError: division by zero
```

try / except

You can catch exceptions and handle them gracefully instead of letting the program crash.

```
try:
    number = int(input("Enter a number: "))
    print(10 / number)
except ValueError:
    print("That wasn't a valid number.")
except ZeroDivisionError:
    print("Cannot divide by zero.")
```

Common built-in exceptions

Exception	Typical cause
ValueError	Right type, wrong value (e.g. <code>int("abc")</code>)
TypeError	Operation on an incompatible type
ZeroDivisionError	Dividing by zero
FileNotFoundError	Trying to open a file that doesn't exist
KeyError	Accessing a dictionary key that doesn't exist
IndexError	Accessing a list index out of range
AttributeError	Calling a method/attribute that doesn't exist on an object

else and finally

```
try:
    result = 10 / 2
```

```
except ZeroDivisionError:
    print("Error!")
else:
    print(f"Success: {result}") # runs only if no exception occurred
finally:
    print("This always runs") # cleanup code, runs no matter what
```

Raising your own exceptions

You can deliberately raise an exception when your code detects an invalid situation.

```
def set_age(age):
    if age < 0:
        raise ValueError("Age cannot be negative")
    return age

try:
    set_age(-5)
except ValueError as e:
    print(f"Invalid input: {e}")
```

TIP: Catch specific exceptions (like `ValueError`) rather than a bare `except:`. A bare `except` silently swallows any error, including typos and bugs, making them much harder to find.

Chapter Summary

- `try / except` catches errors so your program can respond instead of crashing.
- Catch specific exception types whenever possible.
- `finally` runs cleanup code regardless of whether an error occurred.
- `raise` lets you signal your own error conditions.

Try It Yourself — Chapter 8

- 1 Write a program that asks the user for two numbers and divides them, handling both `ValueError` and `ZeroDivisionError` gracefully.
- 2 Write a function `safe_get(dictionary, key)` that returns `None` instead of raising a `KeyError` when the key is missing.
- 3 Write a function that validates an age input, raising a `ValueError` with a clear message if the age is negative or over 130.

Object-Oriented Programming

What is a class?

A class is a blueprint for creating objects that bundle together data (attributes) and behavior (methods). Object-oriented programming (OOP) helps organize larger programs around real-world concepts.

```
class Dog:
    def __init__(self, name, breed):
        self.name = name
        self.breed = breed

    def bark(self):
        print(f"{self.name} says Woof!")

my_dog = Dog("Rex", "Labrador")
print(my_dog.name)    # Rex
my_dog.bark()         # Rex says Woof!
```

Breaking it down

- `class Dog:` defines a new type called Dog.
- `__init__` is the constructor — it runs automatically when you create a new object, setting up its initial attributes.
- `self` refers to the specific object being worked with — always the first parameter of a method.
- `my_dog = Dog("Rex", "Labrador")` creates an instance (an actual object) of the Dog class.

Instance attributes vs. class attributes

```
class Dog:
    species = "Canis familiaris"    # class attribute – shared by all dogs

    def __init__(self, name):
        self.name = name            # instance attribute – unique per dog

dog1 = Dog("Rex")
dog2 = Dog("Bella")
print(dog1.species, dog2.species)  # same for both
print(dog1.name, dog2.name)        # different
```

Methods that use and change state

```
class BankAccount:
    def __init__(self, owner, balance=0):
        self.owner = owner
```

```

        self.balance = balance

    def deposit(self, amount):
        self.balance += amount

    def withdraw(self, amount):
        if amount > self.balance:
            print("Insufficient funds")
        else:
            self.balance -= amount

    def __str__(self):
        return f"{self.owner}'s balance: ${self.balance}"

account = BankAccount("Sam", 100)
account.deposit(50)
account.withdraw(30)
print(account)    # Sam's balance: $120

```

NOTE: `__str__` is a special "dunder" (double-underscore) method that controls what `print()` shows for an object. Python has many such special methods for customizing built-in behavior.

Inheritance

Inheritance lets one class reuse and extend the behavior of another, reducing duplicate code.

```

class Animal:
    def __init__(self, name):
        self.name = name

    def speak(self):
        print(f"{self.name} makes a sound")

class Cat(Animal):
    def speak(self):
        print(f"{self.name} says Meow!")

class Fish(Animal):
    pass    # inherits speak() unchanged

cat = Cat("Whiskers")
cat.speak()    # Whiskers says Meow!

fish = Fish("Nemo")
fish.speak()    # Nemo makes a sound

```

Cat and Fish both inherit from Animal. Cat overrides `speak()` with its own version; Fish simply uses the parent's version.

Properties: controlling attribute access

Sometimes you want an attribute to be computed on the fly, or you want to validate a value before it's stored. The `@property` decorator lets a method be accessed like a plain attribute.

```
class Circle:
    def __init__(self, radius):
        self._radius = radius

    @property
    def area(self):
        return 3.14159 * self._radius ** 2

    @property
    def radius(self):
        return self._radius

    @radius.setter
    def radius(self, value):
        if value < 0:
            raise ValueError("Radius cannot be negative")
        self._radius = value

c = Circle(5)
print(c.area)    # 78.53975 – computed, not stored

c.radius = 10    # goes through the setter, validated
print(c.area)    # updates automatically
```

More special ("dunder") methods

Beyond `__init__` and `__str__`, Python lets classes customize many built-in behaviors:

Method	Customizes
<code>__str__</code>	<code>print(obj)</code> and <code>str(obj)</code> — human-readable text
<code>__repr__</code>	How <code>obj</code> appears in the interactive shell / debugger
<code>__len__</code>	<code>len(obj)</code>
<code>__eq__</code>	<code>obj1 == obj2</code>
<code>__lt__</code>	<code>obj1 < obj2</code> (enables sorting)
<code>__add__</code>	<code>obj1 + obj2</code>

```
class Money:
    def __init__(self, amount):
        self.amount = amount

    def __add__(self, other):
        return Money(self.amount + other.amount)

    def __eq__(self, other):
```

```
        return self.amount == other.amount

    def __str__(self):
        return f"${self.amount:.2f}"

wallet = Money(20) + Money(5.50)
print(wallet)                # $25.50
print(Money(10) == Money(10)) # True
```

Why use OOP?

- Groups related data and behavior together, mirroring real-world entities.
- Encourages reuse through inheritance instead of copy-pasting code.
- Makes large codebases easier to navigate and extend over time.

Chapter Summary

- Classes are blueprints; objects (instances) are created from them.
- `__init__` sets up new objects; `self` refers to the current instance.
- Inheritance lets a subclass reuse and override behavior from a parent class.

Try It Yourself — Chapter 9

- 1 Create a `Rectangle` class with `width` and `height` attributes and methods `area()` and `perimeter()`.
- 2 Create a `Vehicle` base class and two subclasses, `Car` and `Motorcycle`, each overriding a `describe()` method differently.
- 3 Add a method to the `BankAccount` class from this chapter called `transfer(other_account, amount)` that moves money between two accounts.

Modules, Packages, and the Standard Library

Why modules?

As programs grow, it helps to split code across multiple files. A module is simply a .py file that can be imported and reused elsewhere.

Importing modules

```
import math
print(math.sqrt(16))    # 4.0
print(math.pi)         # 3.14159...

from math import sqrt, pi
print(sqrt(25))         # 5.0

import math as m
print(m.sqrt(9))        # 3.0
```

Writing your own module

Suppose you save the following as helpers.py:

```
# helpers.py
def greet(name):
    return f"Hello, {name}!"

PI = 3.14159
```

You can then use it from another file in the same folder:

```
# main.py
import helpers

print(helpers.greet("Sam"))
print(helpers.PI)
```

Handy modules from the standard library

Module	What it's for
math	Mathematical functions (sqrt, floor, trig, constants)
random	Generating random numbers and choices
datetime	Working with dates and times
os	Interacting with the operating system and file paths

Module	What it's for
sys	Interacting with the Python interpreter itself
json	Reading and writing JSON data
re	Regular expressions for pattern matching in text
collections	Extra data structures (Counter, defaultdict, etc.)

A few quick examples

```
import random
print(random.randint(1, 6))          # random int between 1 and 6
print(random.choice(["a", "b", "c"])) # random item from a list

import datetime
now = datetime.datetime.now()
print(now.year, now.month, now.day)
```

Installing third-party packages with pip

The standard library covers a lot, but the wider Python ecosystem has many more packages installable with pip, Python's package manager.

```
pip install requests
pip install pandas
pip list    # see installed packages

import requests
response = requests.get("https://api.github.com")
print(response.status_code)
```

NOTE: It's good practice to use a virtual environment (`python -m venv env`) for each project, so packages installed for one project don't conflict with another. This becomes more important as you work on multiple projects.

Pattern matching with the re module

Regular expressions (regex) describe text patterns — useful for validating input, extracting data, or searching text in flexible ways.

```
import re

text = "Contact us at support@example.com or sales@example.com"
emails = re.findall(r"[\w.]+@[ \w.]+", text)
print(emails)    # ['support@example.com', 'sales@example.com']

if re.match(r"^ \d{3}- \d{4}$", "555-1234"):
    print("Valid phone format")
```

Pattern	Matches
\d	Any digit
\w	Any word character (letter, digit, underscore)
+	One or more of the previous item
*	Zero or more of the previous item
^ \$	Start / end of string

Command-line scripts with argparse

If you write scripts meant to be run from the terminal with options, argparse handles parsing arguments and generates helpful usage messages automatically.

```
# greet.py
import argparse

parser = argparse.ArgumentParser(description="Greet someone")
parser.add_argument("name", help="Name of the person to greet")
parser.add_argument("--loud", action="store_true", help="Shout the greeting")
args = parser.parse_args()

greeting = f"Hello, {args.name}!"
print(greeting.upper() if args.loud else greeting)

$ python greet.py Sam
Hello, Sam!

$ python greet.py Sam --loud
HELLO, SAM!
```

Chapter Summary

- Modules are .py files you can import to reuse code.
- The standard library ships with Python and covers dates, math, randomness, files, and more.
- pip install adds third-party packages beyond the standard library.

Try It Yourself — Chapter 10

- 1 Write a small module called `calculator.py` with `add`, `subtract`, `multiply`, and `divide` functions, then import and use it from a separate script.
- 2 Use the `random` module to simulate rolling two six-sided dice 10,000 times and print how often each total (2 through 12) occurred.
- 3 Use the `datetime` module to print how many days remain until January 1st of next year.

Iterators, Generators, and Functional Tools

What makes something "iterable"?

Anything you can loop over with a for loop — strings, lists, tuples, dictionaries, sets, and files — is called iterable. Under the hood, Python asks the object for an iterator, an object that produces one value at a time until it's exhausted.

```
numbers = [1, 2, 3]
iterator = iter(numbers)
print(next(iterator))  # 1
print(next(iterator))  # 2
print(next(iterator))  # 3
# next(iterator) again would raise StopIteration
```

A for loop does this automatically behind the scenes, which is why it works the same way on so many different types.

Generator functions

A generator is a special kind of function that produces values one at a time, on demand, instead of building an entire list in memory up front. It uses `yield` instead of `return`.

```
def count_up_to(n):
    count = 1
    while count <= n:
        yield count
        count += 1

for number in count_up_to(5):
    print(number)  # 1 2 3 4 5, one at a time
```

Each call to `yield` pauses the function and hands a value back to the caller; the function resumes right where it left off the next time a value is requested.

NOTE: Generators are memory-efficient for large or infinite sequences, because they never hold the whole sequence in memory at once — only the current value and enough state to compute the next one.

Generator expressions

Similar to a list comprehension, but built with parentheses instead of square brackets, and lazy (values are computed only as needed).

```
squares_list = [n ** 2 for n in range(1000000)]  # builds the whole list in memory now
squares_gen = (n ** 2 for n in range(1000000))  # computes each value only when asked

total = sum(n ** 2 for n in range(10))
```

```
print(total)    # 285
```

map(), filter(), and sorted() with key functions

These built-in functions apply logic across a sequence without writing an explicit loop.

```
numbers = [1, 2, 3, 4, 5]

doubled = list(map(lambda n: n * 2, numbers))
print(doubled)    # [2, 4, 6, 8, 10]

evens = list(filter(lambda n: n % 2 == 0, numbers))
print(evens)      # [2, 4]

people = [("Sam", 25), ("Ana", 19), ("Lee", 31)]
by_age = sorted(people, key=lambda person: person[1])
print(by_age)     # sorted by age
```

TIP: A list comprehension is usually considered more "Pythonic" (idiomatic) than map()/filter() with a lambda — both work, but comprehensions tend to read more clearly to most Python programmers.

Dictionary and set comprehensions

Comprehension syntax also works for dictionaries and sets, not just lists.

```
names = ["Alice", "Bob", "Carol"]

name_lengths = {name: len(name) for name in names}
print(name_lengths)    # {'Alice': 5, 'Bob': 3, 'Carol': 5}

unique_lengths = {len(name) for name in names}
print(unique_lengths)  # {3, 5}
```

Unpacking and the * operator

```
first, *rest = [1, 2, 3, 4, 5]
print(first)    # 1
print(rest)     # [2, 3, 4, 5]

a, b, *middle, z = [1, 2, 3, 4, 5, 6]
print(a, b, middle, z)    # 1 2 [3, 4, 5] 6

def total(*args):
    return sum(args)

numbers = [1, 2, 3]
print(total(*numbers))    # unpacks list into separate arguments -> 6
```

Chapter Summary

- Iterables produce values one at a time through an underlying iterator.
- Generators (yield) produce values lazily, saving memory for large sequences.
- `map()`, `filter()`, and `sorted(key=...)` apply logic across sequences concisely.
- The `*` operator unpacks sequences into individual values or function arguments.

Try It Yourself — Chapter 11

- 1 Write a generator function `even_numbers(limit)` that yields even numbers from 0 up to `limit`.
- 2 Use `sorted()` with a key function to sort a list of words by length, shortest first.
- 3 Use a dictionary comprehension to build a dictionary mapping each number from 1 to 10 to its cube.
- 4 Rewrite a `map()/lambda` expression of your choice as an equivalent list comprehension.

Debugging, Testing, and Writing Clean Code

Reading tracebacks

When a program crashes, Python prints a traceback — a report showing exactly where the error happened. Beginners often panic at the wall of text, but the most useful information is almost always at the very bottom.

```
def divide(a, b):
    return a / b

divide(10, 0)

Traceback (most recent call last):
  File "script.py", line 3, in <module>
    divide(10, 0)
  File "script.py", line 2, in divide
    return a / b
ZeroDivisionError: division by zero
```

- The last line tells you the error type and message — start here.
- The lines above show the call stack — which line called which, working backward from where the crash happened.
- The line number and filename tell you exactly where to look in your code.

Using print() to debug

The simplest debugging technique is also one of the most effective: print out variable values at key points to see what your program is actually doing versus what you expected.

```
def calculate_total(prices):
    total = 0
    for price in prices:
        print(f"Adding {price}, running total: {total}") # debug line
        total += price
    return total
```

TIP: Remove or comment out debug print statements once you've fixed the issue — leaving them in clutters your program's real output.

Using the debugger

Most editors (VS Code, PyCharm) include a built-in debugger that lets you pause execution at a specific line (a breakpoint) and inspect variables interactively, which is far more powerful than scattering print statements. It's worth learning your editor's debugger once your programs grow beyond a few dozen lines.

Common beginner mistakes

Mistake	Fix
Using = instead of == in a condition	= assigns, == compares; Python usually errors on this one
Off-by-one errors in range() or slicing	Remember range(n) stops before n
Inconsistent indentation	Use 4 spaces consistently; configure your editor to insert spaces, not tabs
Modifying a list while looping over it	Loop over a copy: for x in list(mylist):
Comparing floats with ==	Floats can have tiny rounding errors – use abs(a - b) < 0.0001 instead

Writing simple tests

A test is code that checks whether other code behaves correctly. Python's built-in assert statement is a lightweight starting point.

```
def add(a, b):  
    return a + b  
  
assert add(2, 3) == 5  
assert add(-1, 1) == 0  
print("All tests passed!")
```

As programs grow, the standard library's unittest module (or the popular third-party pytest package) organizes many such checks into a proper test suite:

```
import unittest  
  
def add(a, b):  
    return a + b  
  
class TestMathFunctions(unittest.TestCase):  
    def test_add_positive(self):  
        self.assertEqual(add(2, 3), 5)  
  
    def test_add_negative(self):  
        self.assertEqual(add(-1, -1), -2)  
  
if __name__ == "__main__":  
    unittest.main()
```

Writing readable code

Code is read far more often than it is written. A few habits go a long way:

- Use descriptive names: customer_count instead of cc or x.

- Keep functions short and focused on one task — if you struggle to name a function without using "and", consider splitting it.
- Follow PEP 8, Python's official style guide (4-space indents, snake_case names, blank lines between functions) — most editors can auto-format this for you.
- Avoid deeply nested if/for blocks; consider early returns to reduce nesting.

```
# Harder to read
def check(age):
    if age >= 0:
        if age < 18:
            return "minor"
        else:
            return "adult"
    else:
        return "invalid"
```

```
# Easier to read – early returns flatten the logic
def check(age):
    if age < 0:
        return "invalid"
    if age < 18:
        return "minor"
    return "adult"
```

Chapter Summary

- Read tracebacks bottom-up: error type/message first, then trace the call stack upward.
- print() debugging and editor breakpoints are both valuable tools.
- assert and unittest help you verify code behaves as expected, especially as it grows.
- Readable code favors clear names, short functions, and flat (not deeply nested) logic.

Try It Yourself — Chapter 12

- 1 Take one of your earlier exercise solutions and add at least 3 assert statements that check it behaves correctly.
- 2 Intentionally introduce an off-by-one bug into a loop, observe the wrong output, then fix it.
- 3 Rewrite a nested if/else block from an earlier chapter using early returns to flatten it.

Mini Projects — Putting It All Together

Reading is not enough — real learning happens when you build things. Below are several small projects that combine ideas from every previous chapter. Try to build each one yourself before looking at the solution code.

Project 1: Number Guessing Game

Goal: the computer picks a random number; the player guesses until they get it right, with hints along the way.

```
import random

def play_game():
    secret = random.randint(1, 100)
    attempts = 0
    print("I'm thinking of a number between 1 and 100.")

    while True:
        guess = int(input("Your guess: "))
        attempts += 1

        if guess < secret:
            print("Too low!")
        elif guess > secret:
            print("Too high!")
        else:
            print(f"Correct! It took you {attempts} attempts.")
            break

    play_game()
```

Concepts used

Modules (random), functions, while loops, conditionals, input/output, type conversion.

Project 2: To-Do List Manager

Goal: a command-line app that lets a user add, view, and remove tasks, saved to a file so the list persists between runs.

```
import json
import os

FILENAME = "tasks.json"

def load_tasks():
```

```

    if os.path.exists(FILENAME):
        with open(FILENAME, "r") as file:
            return json.load(file)
    return []

def save_tasks(tasks):
    with open(FILENAME, "w") as file:
        json.dump(tasks, file)

def show_tasks(tasks):
    if not tasks:
        print("No tasks yet!")
    for i, task in enumerate(tasks, start=1):
        print(f"{i}. {task}")

def main():
    tasks = load_tasks()
    while True:
        print("\n1) Add 2) View 3) Remove 4) Quit")
        choice = input("Choose an option: ")

        if choice == "1":
            task = input("New task: ")
            tasks.append(task)
            save_tasks(tasks)
        elif choice == "2":
            show_tasks(tasks)
        elif choice == "3":
            show_tasks(tasks)
            index = int(input("Number to remove: ")) - 1
            if 0 <= index < len(tasks):
                tasks.pop(index)
                save_tasks(tasks)
        elif choice == "4":
            break
        else:
            print("Invalid option")

    main()

```

Concepts used

Functions, file I/O, JSON, lists, loops, conditionals, error-prone user input handling.

Project 3: Simple Contact Book (using classes)

```

class Contact:
    def __init__(self, name, phone, email):
        self.name = name
        self.phone = phone
        self.email = email

```

```

def __str__(self):
    return f"{self.name} | {self.phone} | {self.email}"

class ContactBook:
    def __init__(self):
        self.contacts = []

    def add(self, contact):
        self.contacts.append(contact)

    def find(self, name):
        for contact in self.contacts:
            if contact.name.lower() == name.lower():
                return contact
        return None

    def list_all(self):
        for contact in self.contacts:
            print(contact)

book = ContactBook()
book.add(Contact("Alex", "555-1234", "alex@example.com"))
book.add(Contact("Priya", "555-5678", "priya@example.com"))
book.list_all()

found = book.find("alex")
print(found)

```

Concepts used

Classes, objects, encapsulating related data, custom `__str__`, list operations.

Project 4: Word Frequency Counter

Goal: read a text file and report how many times each word appears.

```

from collections import Counter

def word_frequencies(filename):
    with open(filename, "r") as file:
        text = file.read().lower()

    words = text.split()
    cleaned = [word.strip(".,!?\"';:") for word in words]
    return Counter(cleaned)

frequencies = word_frequencies("sample.txt")
for word, count in frequencies.most_common(5):
    print(f"{word}: {count}")

```

Concepts used

File reading, string methods, list comprehensions, the collections module.

Project 5: Multiple-Choice Quiz App

Goal: load quiz questions from a list of dictionaries, ask the user each one, and report a final score.

```
questions = [
    {"question": "What does len() do?",
     "options": ["Adds items", "Returns length", "Deletes items"], "answer": 1},
    {"question": "Which keyword defines a function?",
     "options": ["func", "def", "define"], "answer": 1},
    {"question": "What symbol starts a comment?",
     "options": ["//", "#", "--"], "answer": 1},
]

def run_quiz(questions):
    score = 0
    for q in questions:
        print(f"\n{q['question']}")
        for i, option in enumerate(q["options"]):
            print(f" {i}) {option}")

        answer = int(input("Your answer (number): "))
        if answer == q["answer"]:
            print("Correct!")
            score += 1
        else:
            correct_text = q["options"][q["answer"]]
            print(f"Wrong – the answer was: {correct_text}")

    print(f"\nFinal score: {score}/{len(questions)}")

run_quiz(questions)
```

Concepts used

Lists of dictionaries, nested loops, enumerate, input validation, functions, string formatting — a good synthesis of nearly everything in this guide.

TIP: Try extending each project: add error handling to the guessing game for non-numeric input, add "edit task" to the to-do app, add a search-by-phone feature to the contact book, save high scores for the quiz app to a JSON file, or add a difficulty setting that changes the question set. Extending existing code is one of the fastest ways to build real skill.

Where to Go Next

You now have a solid foundation covering the core of the Python language. Here's how to keep building from here.

Practice platforms

- **Exercism.org** — bite-sized exercises with mentor feedback, free.
- **LeetCode / HackerRank** — algorithm and data-structure practice, useful before technical interviews.
- **Advent of Code** — a fun yearly programming puzzle series, great for building problem-solving stamina.

Pick a direction

Interest	Learn next
Web development	Flask or Django (backend), then HTML/CSS/JavaScript basics
Data analysis	pandas, NumPy, matplotlib/seaborn, then Jupyter notebooks
Automation/scripting	os, shutil, argparse, and writing small CLI tools
Machine learning	NumPy and pandas first, then scikit-learn, then PyTorch/TensorFlow
Game development	pygame for 2D games
APIs and backend services	requests, FastAPI or Flask, and basic SQL

Habits that accelerate learning

- 1 Build small real projects instead of only doing exercises — a project you actually want to finish keeps you motivated through the boring parts.
- 2 Read other people's code, especially on well-maintained open-source projects on GitHub.
- 3 Get comfortable reading error messages and tracebacks — they usually tell you exactly what went wrong and on which line.
- 4 Use version control (git) early, even for small personal projects.
- 5 Write a little bit of code every day rather than occasional long sessions — consistency beats intensity for skill-building.

20 more project ideas to build real skill

Once you've finished the projects in Chapter 13, here are more ideas roughly ordered from easier to harder. Pick ones that sound fun — motivation matters more than difficulty level.

- 1 Unit converter (length, weight, temperature) with a menu
- 2 Simple calculator that parses expressions like "3 + 4 * 2"
- 3 Rock-paper-scissors game against the computer
- 4 Password strength checker using string and re rules
- 5 Random password generator with customizable length and character sets
- 6 Text-based adventure game with rooms and choices stored in dictionaries
- 7 Flashcard quiz app that tracks which cards you get wrong most often
- 8 Simple expense tracker that saves entries to CSV and reports monthly totals
- 9 Markdown-to-HTML converter for a few basic tags (headers, bold, links)
- 10 Command-line dice roller supporting notation like "2d6+3"
- 11 Basic Tic-Tac-Toe game for two players in the terminal
- 12 Weather lookup tool using the requests library and a free weather API
- 13 Simple web scraper that pulls headlines from a news site (respecting robots.txt)
- 14 Personal library catalog: add/search/remove books, saved to JSON
- 15 Basic Conway's Game of Life simulation printed to the terminal
- 16 Recipe scaler that adjusts ingredient quantities for a different serving size
- 17 Habit tracker that logs daily check-ins to a file and shows a streak count
- 18 Simple Caesar cipher encoder/decoder for text messages
- 19 Command-line Pomodoro timer with work/break cycles
- 20 Basic budget calculator comparing planned vs. actual spending by category

Final note

Every experienced programmer was once a beginner who got stuck on things that now seem obvious — a missing colon, a misplaced indent, confusing = with ==. That struggle is normal and is literally how the skill is built. The single best next step is to close this PDF and go write some code.

APPENDIX

Virtual Environments in More Detail

A virtual environment is an isolated Python installation for a single project, so its installed packages don't conflict with other projects on your machine. This matters more than it might seem: two projects needing different versions of the same package will conflict without one.

```
# Create a virtual environment named 'env' in the current folder
python -m venv env

# Activate it (Windows)
env\Scripts\activate

# Activate it (macOS/Linux)
source env/bin/activate

# Once activated, installs are isolated to this project
pip install requests

# Deactivate when done
deactivate
```

TIP: Your terminal prompt usually shows the environment name (like (env)) when a virtual environment is active — a quick visual check that you're in the right place before installing packages.

APPENDIX

Basic Version Control with Git

Git tracks changes to your code over time, letting you undo mistakes, see history, and collaborate with others. It's a separate tool from Python but essential for any real project, even solo ones.

Command	What it does
git init	Start tracking a folder as a Git repository
git add <file>	Stage a file's changes to be included in the next commit
git commit -m "message"	Save a snapshot of staged changes with a description
git status	Show what's changed since the last commit
git log	View commit history
git push	Upload commits to a remote repository (e.g. GitHub)
git clone <url>	Download a copy of a remote repository

A typical workflow: make changes to your code, run `git add .` to stage everything, then `git commit -m "describe what changed"` to save a snapshot. Repeat as you work.

Common Gotchas for Beginners Moving to Intermediate Code

Gotcha	Explanation
Mutable default arguments	<code>def f(items=[]):</code> reuses the same list across calls – use <code>None</code> and create the list inside the function instead
Copying lists	<code>new_list = old_list</code> makes both names point to the same list; use <code>new_list = old_list.copy()</code> or <code>new_list = old_list[:]</code>
Integer division surprises	<code>5 / 2</code> gives 2.5 in Python 3 (not 2, like some older languages) – use <code>//</code> for floor division
Late binding in closures	Lambdas inside a loop can all reference the loop variable's final value unless you capture it explicitly (e.g. via a default argument)
String vs. list of characters	<code>"abc"[0]</code> works like indexing a list, but strings can't be modified in place the way lists can

```
# Mutable default argument pitfall
def add_item(item, items=[]): # BAD: shared across calls
    items.append(item)
    return items

def add_item(item, items=None): # GOOD
    if items is None:
        items = []
    items.append(item)
    return items
```

Quick reference cheat sheet

A condensed reference of syntax covered in this guide, for quick lookups while you code.

Task	Syntax
Variable assignment	<code>x = 5</code>
f-string	<code>f"{name} is {age}"</code>
if/elif/else	<code>if x > 0: ... elif x == 0: ... else: ...</code>
for loop	<code>for item in sequence: ...</code>
while loop	<code>while condition: ...</code>
Define a function	<code>def name(params): ... return value</code>

Task	Syntax
List	<code>my_list = [1, 2, 3]</code>
Dictionary	<code>my_dict = {"key": "value"}</code>
List comprehension	<code>[x*2 for x in range(5)]</code>
Try/except	<code>try: ... except ValueError: ...</code>
Define a class	<code>class Name: def __init__(self, ...): ...</code>
Open a file	<code>with open("file.txt") as f: ...</code>
Import a module	<code>import module_name</code>

End of guide. Happy coding!